

Tech Challenge - Fase 2

Migração do ToggleMaster para Microsserviços em Kubernetes na AWS

Pós Tech - FIAP

Nome	RM	Discord
Lucas Gabriel E. Campos	RM371200	Campalô
Rafael Marques Muniz	RM373742	KaollanRafa
Edson Maciel do Vale Junior	RM372784	EJR_04#8240
João Pedro Soares Oliveira	RM371227	Jotape
Matheus Villão Gonçalves	RM370682	villao.

Repositório GitHub: <https://github.com/mathvillao/togglmaster-tech-challenge-fase-2.git>

Vídeo demonstrativo: <https://youtu.be/NDFY0PQ9oQs?si=ww0zMjZN6zacrQ2S>

1. Resumo do Projeto

O objetivo deste Tech Challenge foi migrar o ToggleMaster de uma arquitetura monolítica para uma arquitetura baseada em microsserviços, com containerização via Docker, orquestração em Kubernetes e implantação em ambiente AWS utilizando Amazon EKS.

Microsserviço	Linguagem	Responsabilidade	Dependências
auth-service	Go	Gerenciamento e validação de chaves de API	PostgreSQL
flag-service	Python/Flask	Cadastro e consulta de feature flags	PostgreSQL
targeting-service	Python/Flask	Cadastro e consulta de regras de segmentação	PostgreSQL
evaluation-service	Go	Avaliação de flags para usuários	Redis, SQS, flag-service, targeting-service
analytics-service	Python/Flask	Consumo de eventos e persistência analítica	SQS, DynamoDB

2. Arquitetura da Solução

A arquitetura final utiliza o Amazon EKS como ambiente principal de execução dos microsserviços. O acesso externo ocorre por meio de um Load Balancer provisionado pelo Nginx Ingress Controller, que roteia as chamadas por path para cada serviço interno do cluster.

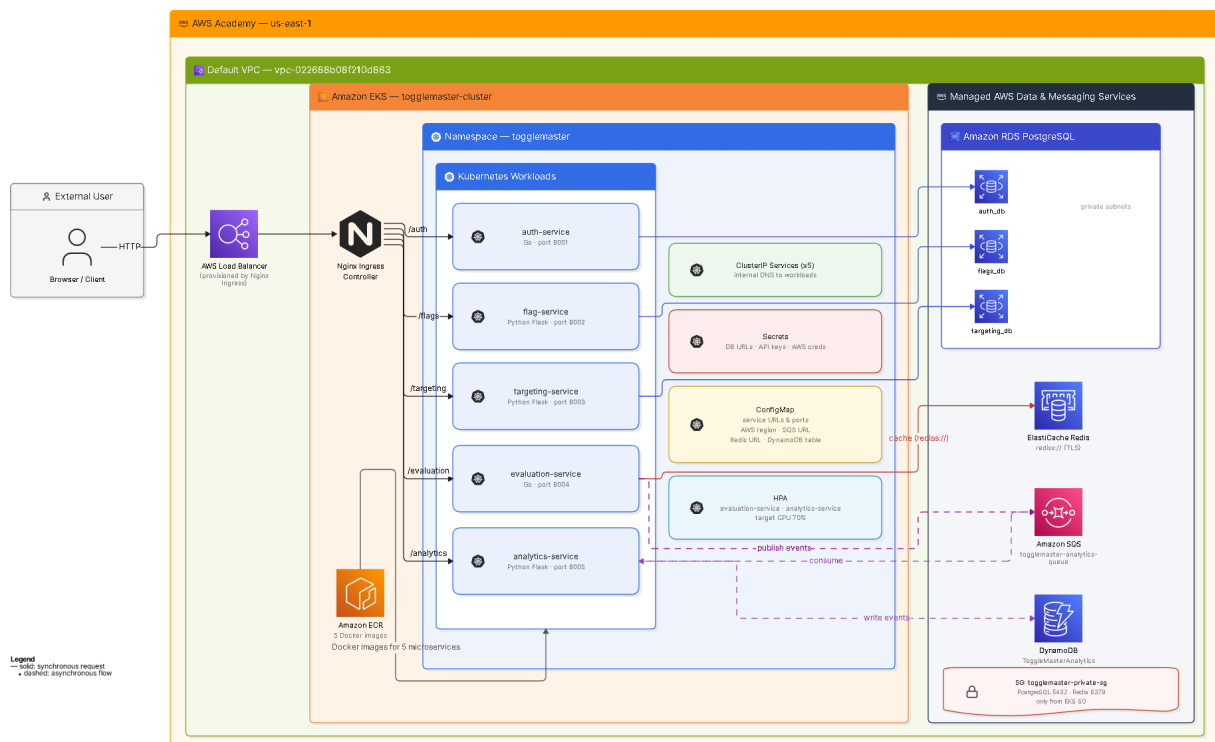


Figura 1 - Arquitetura da solução ToggleMaster em microsserviços no AWS EKS.

- Cada microsserviço possui seu próprio container e manifesto Kubernetes.
- Os serviços internos usam ClusterIP, mantendo comunicação privada dentro do cluster.
- O acesso externo é centralizado pelo Ingress.

- Dados transacionais foram separados em três bancos PostgreSQL no Amazon RDS.
- Redis foi provisionado no Amazon ElastiCache para cache de avaliações.
- O fluxo analítico foi desacoplado com Amazon SQS e persistido no Amazon DynamoDB.
- Configurações foram separadas em ConfigMap e dados sensíveis em Secret.
- evaluation-service e analytics-service possuem HPA com alvo de CPU em 70%.

3. Containerização e Ambiente Local

Foram criados Dockerfiles para os cinco microsserviços. Os serviços em Go utilizam build multi-stage, reduzindo o tamanho da imagem final. Os serviços Python utilizam imagens Alpine atualizadas e executam com usuário não-root.

- Docker Compose com nove containers: cinco aplicações e quatro recursos locais.
- Dois PostgreSQL locais, Redis e DynamoDB Local.
- Health checks dos cinco serviços.
- Validação do fluxo completo: chave de API, feature flag, targeting rule, evaluation e analytics.
- Correção de problemas de build e dependências dos serviços Go e Python.

4. Hardening das Imagens Docker

- Atualização das imagens base para versões mais recentes.
- Uso de python:3.13-alpine3.24 nos serviços Python.
- Uso de golang:1.25-alpine3.24 e alpine:3.24 nos serviços Go.
- Atualização de dependências Python.
- Execução dos containers com usuário não-root.
- Validação com Docker Scout, sem vulnerabilidades críticas ou altas detectadas.

5. Infraestrutura AWS

Serviço AWS	Recurso	Finalidade
Amazon ECR	5 repositórios	Armazenar imagens Docker dos microsserviços
Amazon EKS	1 cluster	Orquestrar os containers Kubernetes
Amazon RDS PostgreSQL	3 instâncias	Bancos para auth, flags e targeting
Amazon ElastiCache Redis	1 cache Redis	Cache do evaluation-service
Amazon DynamoDB	1 tabela	Persistência dos eventos analíticos
Amazon SQS	1 fila	Comunicação assíncrona entre evaluation e analytics
Elastic Load Balancer	1 Load Balancer	Entrada HTTP criada pelo Nginx Ingress
Security Groups	Regras privadas	Controle de acesso entre EKS, RDS e Redis

6. Kubernetes e Manifestos

Os manifestos Kubernetes foram organizados na pasta k8s, contendo namespace, secrets de exemplo, ConfigMap, Deployments, Services, Jobs, Ingress e HPAs.

- Cada microsserviço possui Deployment e Service ClusterIP.
- Foram configuradas probes de saúde, requests e limits de CPU/memória.
- Jobs inicializam os bancos auth_db, flags_db e targeting_db.
- Nginx Ingress Controller roteia /auth, /flags, /targeting, /evaluation e /analytics.
- Metrics Server habilita métricas para HPA.

7. Escalabilidade

HPA	Serviço	Métrica	Alvo	Réplicas
evaluation-service-hpa	evaluation-service	CPU	70%	1 a 3
analytics-service-hpa	analytics-service	CPU	70%	1 a 3

Também foi executado teste de carga para demonstrar o comportamento de escalabilidade do evaluation-service.

8. Segurança e Configuração

- Credenciais reais não foram versionadas no repositório.
- Uso de .env.example e 01-secrets.example.yaml.
- Kubernetes Secrets para URLs de banco, API keys e credenciais AWS temporárias.
- ConfigMap para valores não sensíveis.
- RDS e Redis sem exposição pública.
- Security Group permitindo PostgreSQL e Redis somente a partir do Security Group do EKS.
- Containers executando com usuário não-root.

9. Testes Realizados

- Ambiente local validado com docker compose up e health checks.
- Build e execução das cinco imagens Docker.
- Fluxo funcional completo validado localmente e no EKS.
- Pods, Services, Jobs, Ingress e HPA validados com kubectl.
- Health checks externos validados pelo Load Balancer.
- Evento de avaliação enviado ao SQS e persistido no DynamoDB pelo analytics-service.

10. Resultado Final

O ToggleMaster foi executado com sucesso em uma arquitetura de microsserviços no Kubernetes, com imagens armazenadas no ECR e infraestrutura AWS gerenciada para banco, cache, mensageria e analytics.

- Dockerfiles para os cinco microsserviços.
- Docker Compose com os nove containers exigidos.

- ECR com uma imagem por serviço.
- EKS funcional com manifests Kubernetes.
- RDS PostgreSQL, ElastiCache Redis, SQS e DynamoDB.
- Metrics Server, Nginx Ingress Controller e HPA.
- Vídeo demonstrativo e repositório versionado no GitHub.

11. Conclusão

A entrega demonstra a migração do ToggleMaster para uma arquitetura moderna baseada em microsserviços, com separação clara de responsabilidades, infraestrutura gerenciada na AWS, execução em Kubernetes e mecanismos de escalabilidade.

Além dos requisitos obrigatórios, foram aplicadas melhorias de segurança, hardening das imagens, organização dos manifests, validação de vulnerabilidades e documentação detalhada da solução.